

NATIONAL UNIVERSITY OF MODERN LANGUAGES

DATABASE SEMESTER PROJECT

PROJECT NAME: CAPITAL FLOWERS

PROJECT TITLE: ISLAMABAD FLOWER SHOP

SUBMITTED TO:	Naveed Alam
PROJECT:	Capital Flowers — Islamabad Flower Shop Management System
FAKHRULLAH MALIK	
FARMANULLAH KHAN	
HIDAYAT ULLAH	
UMAIR ASLAM	
SHAHEER KHAN	
ADIL SALMAN QAZI	June 30, 2026

TASK 1: ERD DIAGRAM

Capital Flowers — Islamabad Flower Shop: Entity Relationship Diagram

6 Entities (Tables):

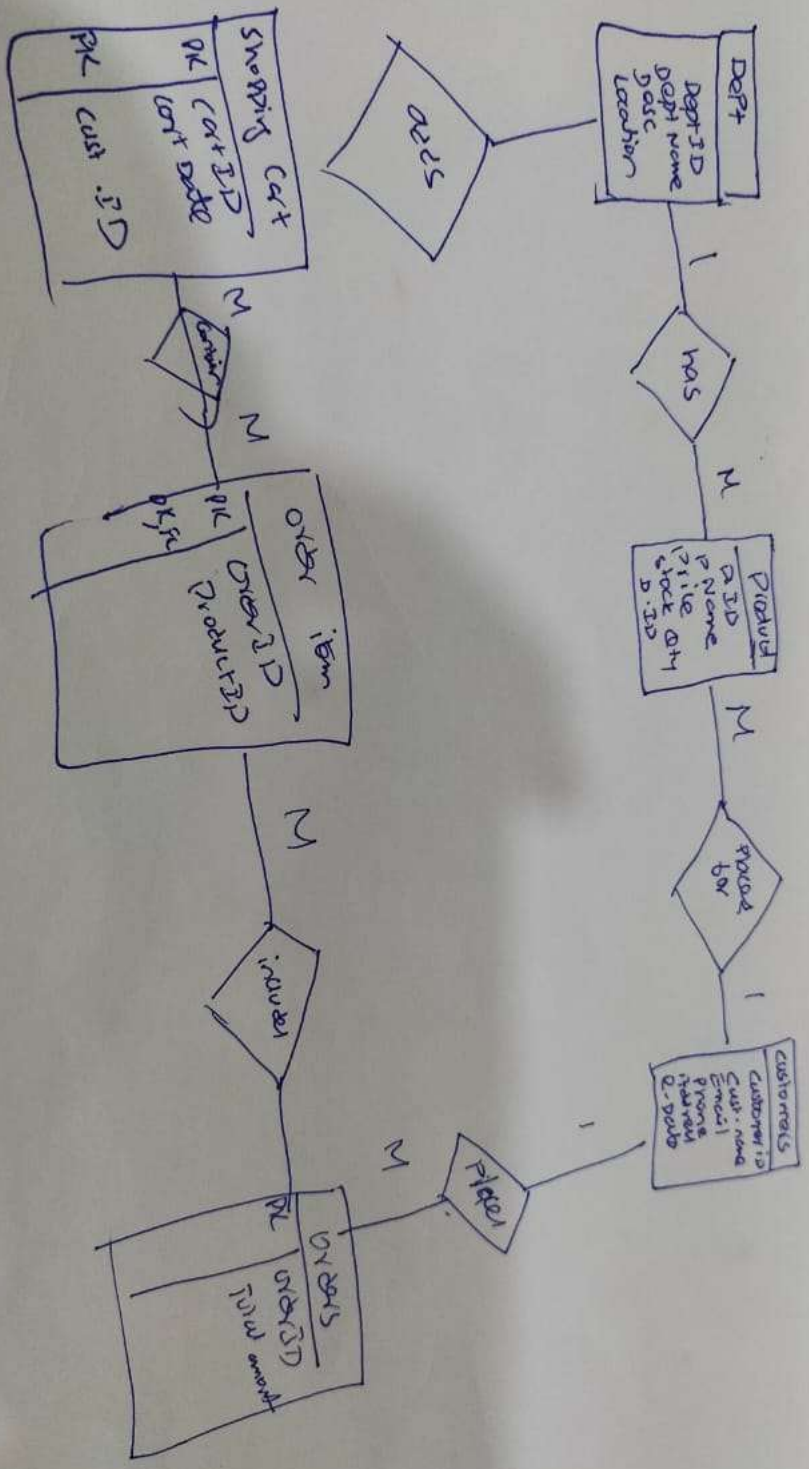
- **Supplier** – companies you buy flowers from (SupplierID PK, SupplierName, ContactNumber)
- **Flower** – the flower catalog (FlowerID PK, CommonName, CostPerStem)
- **Shipment** – one delivery event from one supplier on one date (Composite PK: SupplierID + DeliveryDate)
- **ShipmentDetail** – line items inside a shipment: which flowers, how many stems, and bulk price paid (Composite PK: SupplierID + DeliveryDate + FlowerID)
- **Bouquet** – finished products you sell (BouquetID PK, BouquetName, RetailPrice)
- **BouquetRecipe** – the recipe for each bouquet: which flowers and how many stems go into it (Composite PK: BouquetID + FlowerID)

5 Relationships (all 1:M, shown with crow's foot):

- Supplier $\square$ Shipment: one supplier makes many shipments
- Shipment $\square$ ShipmentDetail: one shipment contains many flower line items
- Flower $\square$ ShipmentDetail: one flower type appears in many shipment line items
- Bouquet $\square$ BouquetRecipe: one bouquet has many recipe lines (one per flower used)
- Flower $\square$ BouquetRecipe: one flower is used in many bouquet recipes

Why junction tables exist: ShipmentDetail and BouquetRecipe are many-to-many resolver tables — they let a single shipment hold multiple flower types, and a single bouquet be made of multiple flower types, while still tracking quantity/price or stem count for each combination.

ERD



TASK 2: NORMALIZATION

Project Entities and System Boundaries

The **Capital Flowers — Islamabad Flower Shop Management System** is an e-commerce platform designed to automate inventory curation, client provisioning, multi-item shopping carts, checkout handling, secure payment tracking, and general customer interactions. The following primary entities have been defined based on system boundaries and functional components:

1. Users

Represents all external registered entities accessing the storefront. Core fields include unique internal identifiers, authenticable email strings, hashed security credentials, personal communications, and physical fulfillment destinations.

2. Admins

Represents privileged administrative accounts authorized to manage back-office logic via the Admin Dashboard including product catalog maintenance, category creation, order status updates, and user inquiries.

3. Categories

Defines the taxonomic categorization of the inventory (e.g., 'Seasonal Bouquets', 'Sympathy Arrangements', 'Indoor Flora') allowing buyers to perform granular product filters.

4. Products

Represents individual SKUs available for purchase — loose flowers, pre-assembled bouquets, customized arrangements, and accessories. Attributes include botanical names, descriptions, pricing, stock metrics, and category references.

5. Orders

Acts as the top-level transactional receipt representing a distinct acquisition commitment by a registered User, recording timestamp, financial liabilities, and execution status.

6. OrderItems

An associative entity representing individual product lines nested within a main Order, tracking distinct quantity and exact frozen price-per-unit assigned during checkout.

7. Payments

Captures structural accounting events tied directly to specific Orders, maintaining historical proof of funds, processing methods, transactional reference codes, and status validations.

8. Contacts

Manages communications submitted through the frontend 'Contact Us' portal — tracking feedback loops, operational queries, and customer assistance tickets.

Functional Dependencies (per table):

1. Users Table

```
UserID → UserName, UserEmail, UserPassword, UserPhone, UserAddress, UserRole, UserCreatedAt
UserEmail → UserID (candidate key)
```

2. Admins Table

```
AdminID □ AdminName, AdminEmail, AdminPassword, SecurityLevel, LastLogin
```

AdminEmail □ AdminID (candidate key)

3. Categories Table

CategoryID □ CategoryName, CategoryDescription

4. Products Table

ProductID → ProductName, ProductDescription, ProductImageURL, ProductPrice, ProductStock,
CategoryID

5. Orders Table

OrderID □ OrderDate, UserID, OrderStatus, TotalAmount

6. OrderItems Table

(OrderID, ProductID) □ Quantity, UnitPrice, Subtotal

7. Payments Table

PaymentID □ OrderID, PaymentDate, PaymentMethod, Amount, PaymentStatus

8. Contacts Table

ContactID □ SenderName, SenderEmail, MsgSubject, MsgBody, SubmittedAt

Unnormalized Form (UNF) — Raw Data Dump

The UNF represents raw unrefined data as it might appear in a flat file or legacy spreadsheet. It contains multi-valued attributes, repeating logical blocks, and complex nested arrays packed into a single master layout called **CapitalFlowers_Master_UNF**.

OrderID	OrderDate	UserID	UserName	UserAddress	ProductID	ProductName	Cat.	Price	Qty	Total	PayMethod	PayStatus
1001	2026-06-25	U902	Ayesha Khan	75 Main Blvd, LHR	P01, P03	Red Roses, White Lilies	Rom., Symp.	45/60	2,1	150.00	Credit Card	Completed
1002	2026-06-26	U411	Bilal Ahmed	12-C, DHA, KHI	P03	White Lilies Bunch	Sympathy	60.00	3	180.00	EasyPaisa	Completed
1003	2026-06-27	U902	Ayesha Khan	75 Main Blvd, LHR	P01, P04	Red Roses, Tulip Basket	Rom., Spr.	45/55	1,2	155.00	COD	Pending

First Normal Form (1NF) — Atomicity Resolution

A relation is in 1NF if all attributes contain exclusively atomic (indivisible) values, with no repeating groups or multi-valued columns. Composite cells are split across distinct rows. The composite primary key becomes **(OrderID, ProductID)**.

OrderID*	ProductID*	OrderDate	UserID	UserName	UserAddress	ProductName	CategoryName	Price	Qty	Subtotal	OrderTotal	PayStatus
1001	P01	2026-06-25	U902	Ayesha Khan	75 Main Blvd, LHR	Crimson Red Roses	Romantic	45.00	2	90.00	150.00	Completed
1001	P03	2026-06-25	U902	Ayesha Khan	75 Main Blvd, LHR	White Lilies Bunch	Sympathy	60.00	1	60.00	150.00	Completed
1002	P03	2026-06-26	U411	Bilal Ahmed	12-C, DHA, KHI	White Lilies Bunch	Sympathy	60.00	3	180.00	180.00	Completed
1003	P01	2026-06-27	U902	Ayesha Khan	75 Main Blvd, LHR	Crimson Red Roses	Romantic	45.00	1	45.00	155.00	Pending
1003	P04	2026-06-27	U902	Ayesha Khan	75 Main Blvd, LHR	Yellow Tulip Basket	Spring	55.00	2	110.00	155.00	Pending

Second Normal Form (2NF) — Partial Dependency Removal

A table is in 2NF if it is in 1NF and eliminates all partial functional dependencies. With composite key (OrderID, ProductID), partial dependencies exist: ProductName/Price depend only on ProductID; OrderDate/UserID depend only on OrderID. Only Qty and Subtotal require both keys. We decompose into three tables:

Table 1: OrderItems_2NF

OrderID*	ProductID*	Qty	Subtotal
1001	P01	2	90.00
1001	P03	1	60.00
1002	P03	3	180.00
1003	P01	1	45.00
1003	P04	2	110.00

Table 2: Products_Intermediate_2NF

ProductID*	ProductName	CategoryID	CategoryName	Price
P01	Crimson Red Roses	C10	Romantic Bouquets	45.00
P03	White Lilies Bunch	C12	Sympathy Floral	60.00
P04	Yellow Tulip Basket	C15	Spring Collection	55.00

Table 3: Orders_Intermediate_2NF

OrderID*	OrderDate	UserID	UserName	UserAddress	OrderTotal	PayMethod	PayStatus
1001	2026-06-25	U902	Ayesha Khan	75 Main Blvd, LHR	150.00	Credit Card	Completed
1002	2026-06-26	U411	Bilal Ahmed	12-C, DHA, KHI	180.00	EasyPaisa	Completed
1003	2026-06-27	U902	Ayesha Khan	75 Main Blvd, LHR	155.00	COD	Pending

Third Normal Form (3NF) — Transitive Dependency Removal

A table is in 3NF if it is in 2NF and contains zero transitive functional dependencies among non-prime attributes ($A \rightarrow B$ and $B \rightarrow C$ means $A \rightarrow C$ is transitive). In `Products_Intermediate_2NF`, `ProductID` $\rightarrow$ `CategoryID` $\rightarrow$ `CategoryName` is transitive. In `Orders_Intermediate_2NF`, `OrderID` $\rightarrow$ `UserID` $\rightarrow$ `UserName/Address` and `OrderID` $\rightarrow$ `PaymentID` $\rightarrow$ `PayMethod/PayStatus` are transitive. We resolve into five clean 3NF tables:

1. Users Table

UserID*	UserName	UserEmail	UserAddress
U902	Ayesha Khan	ayesha@mail.com	75 Main Blvd, LHR
U411	Bilal Ahmed	bilal@mail.com	12-C, DHA, KHI
U303	Zainab Fatima	zainab@mail.com	Sector G-11, ISB
U215	Omar Sheikh	omar@mail.com	F-8/2, Islamabad

2. Categories Table

CategoryID*	CategoryName
C10	Romantic Bouquets
C12	Sympathy Floral
C15	Spring Collection
C18	Indoor Flora

3. Products Table

ProductID*	ProductName	Price	CategoryID (FK)
P01	Crimson Red Roses	45.00	C10
P03	White Lilies Bunch	60.00	C12
P04	Yellow Tulip Basket	55.00	C15
P05	Midnight Orchid Deluxe	85.00	C10
P06	Sunflower Surprise	40.00	C15

4. Orders Table

OrderID*	OrderDate	UserID (FK)	OrderTotal
1001	2026-06-25	U902	150.00
1002	2026-06-26	U411	180.00
1003	2026-06-27	U902	155.00
1004	2026-06-28	U303	85.00
1005	2026-06-29	U215	125.00

5. Payments Table

PaymentID*	OrderID (FK)	PayMethod	PayStatus
PAY-881	1001	Credit Card	Completed

PAY-882	1002	EasyPaisa	Completed
PAY-883	1003	Cash on Delivery	Pending
PAY-884	1004	Credit Card	Completed
PAY-885	1005	EasyPaisa	Pending

TASK 3: SQL SCRIPT

PART 1: DDL — Schema Creation

```
--=====
-- CAPITAL FLOWERS — ISLAMABAD FLOWER SHOP MANAGEMENT SYSTEM
-- TARGET DATABASE: MySQL 8.0+ (InnoDB Engine)
--=====

DROP DATABASE IF EXISTS capital_flowers_dbs;
CREATE DATABASE capital_flowers_dbs
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE capital_flowers_dbs;

-- 1. USERS TABLE
CREATE TABLE users (
  user_id INT AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  phone VARCHAR(20) NOT NULL,
  address TEXT NOT NULL,
  role ENUM('customer','admin','superadmin') DEFAULT 'customer',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT pk_users PRIMARY KEY (user_id),
  CONSTRAINT uq_user_email UNIQUE (email)
) ENGINE=InnoDB;

-- 2. ADMINS TABLE
CREATE TABLE admins (
  admin_id INT AUTO_INCREMENT,
  user_id INT NOT NULL,
  security_level INT NOT NULL DEFAULT 1,
  last_login TIMESTAMP NULL DEFAULT NULL,
  CONSTRAINT pk_admins PRIMARY KEY (admin_id),
  CONSTRAINT fk_admins_users FOREIGN KEY (user_id)
REFERENCES users(user_id) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 3. CATEGORIES TABLE
CREATE TABLE categories (
  category_id INT AUTO_INCREMENT,
  category_name VARCHAR(100) NOT NULL,
  description TEXT NULL,
  CONSTRAINT pk_categories PRIMARY KEY (category_id),
  CONSTRAINT uq_category_name UNIQUE (category_name)
) ENGINE=InnoDB;

-- 4. PRODUCTS TABLE
CREATE TABLE products (
  product_id INT AUTO_INCREMENT,
  product_name VARCHAR(150) NOT NULL,
  description TEXT NULL,
  image_url VARCHAR(255) NULL,
  price DECIMAL(10,2) NOT NULL,
  stock_quantity INT NOT NULL DEFAULT 0,
  category_id INT NOT NULL,
```

```

CONSTRAINT pk_products PRIMARY KEY (product_id),
CONSTRAINT chk_product_price CHECK (price >= 0.00),
CONSTRAINT chk_product_stock CHECK (stock_quantity >= 0),
CONSTRAINT fk_products_categories
FOREIGN KEY (category_id) REFERENCES categories(category_id)
ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 5. ORDERS TABLE
CREATE TABLE orders (
order_id INT AUTO_INCREMENT,
order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
user_id INT NOT NULL,
order_status ENUM('Pending','Processing','Dispatched',
'Delivered','Cancelled') DEFAULT 'Pending',
total_amount DECIMAL(10,2) NOT NULL DEFAULT 0.00,
CONSTRAINT pk_orders PRIMARY KEY (order_id),
CONSTRAINT chk_order_total CHECK (total_amount >= 0.00),
CONSTRAINT fk_orders_users FOREIGN KEY (user_id)
REFERENCES users(user_id) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 6. ORDER_ITEMS TABLE
CREATE TABLE order_items (
order_id INT NOT NULL,
product_id INT NOT NULL,
quantity INT NOT NULL,
unit_price DECIMAL(10,2) NOT NULL,
subtotal DECIMAL(10,2) NOT NULL,
CONSTRAINT pk_order_items PRIMARY KEY (order_id, product_id),
CONSTRAINT chk_item_qty CHECK (quantity > 0),
CONSTRAINT chk_item_price CHECK (unit_price >= 0.00),
CONSTRAINT fk_items_orders FOREIGN KEY (order_id)
REFERENCES orders(order_id) ON DELETE CASCADE ON UPDATE CASCADE,
CONSTRAINT fk_items_products FOREIGN KEY (product_id)
REFERENCES products(product_id) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 7. PAYMENTS TABLE
CREATE TABLE payments (
payment_id INT AUTO_INCREMENT,
order_id INT NOT NULL,
payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
payment_method VARCHAR(50) NOT NULL,
amount DECIMAL(10,2) NOT NULL,
payment_status ENUM('Pending','Completed','Failed','Refunded')
DEFAULT 'Pending',
CONSTRAINT pk_payments PRIMARY KEY (payment_id),
CONSTRAINT chk_payment_amt CHECK (amount >= 0.00),
CONSTRAINT fk_payments_orders FOREIGN KEY (order_id)
REFERENCES orders(order_id) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 8. CONTACTS TABLE
CREATE TABLE contacts (
contact_id INT AUTO_INCREMENT,

```

```

sender_name VARCHAR(100) NOT NULL,
sender_email VARCHAR(150) NOT NULL,
subject VARCHAR(200) NOT NULL,
message_body TEXT NOT NULL,
submitted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT pk_contacts PRIMARY KEY (contact_id)
) ENGINE=InnoDB;

-- PERFORMANCE INDEXES

CREATE INDEX idx_products_name ON products(product_name);
CREATE INDEX idx_orders_user ON orders(user_id);
CREATE INDEX idx_payments_order ON payments(order_id);

```

PART 2: DML — Sample Data Population

```

-- 1. USERS
INSERT INTO users (user_id, name, email, password_hash, phone, address, role, created_at)
VALUES
(1,'Admin Master','admin@capitalflowers.com','$2y$10$hash1...',
'0300-1112223','HQ Office, Blue Area, ISB','admin','2026-01-10 09:00:00'),
(2,'Ayesha Khan','ayasha@mail.com','$2y$10$hash2...',
'0321-5554433','75 Main Blvd, Gulberg, LHR','customer','2026-06-20 14:22:11'),
(3,'Bilal Ahmed','bilal@mail.com','$2y$10$hash3...',
'0333-9998877','12-C, Phase 5, DHA, KHI','customer','2026-06-21 11:05:45'),
(4,'Zainab Fatima','zainab@mail.com','$2y$10$hash4...',
'0345-4443322','Sector G-11, Street 4, ISB','customer','2026-06-24 18:30:00'),
(5,'Omar Sheikh','omar@mail.com','$2y$10$hash5...',
'0312-7776655','F-8/2, Islamabad','customer','2026-06-25 10:00:00');

-- 2. ADMINS
INSERT INTO admins (admin_id, user_id, security_level, last_login) VALUES
(1, 1, 5, '2026-06-29 08:00:00');

-- 3. CATEGORIES
INSERT INTO categories (category_id, category_name, description) VALUES
(10,'Romantic Bouquets',
'Elegant collections featuring premium imported roses and custom wrappers.'),
(12,'Sympathy Floral',
'Serene arrangements using white lilies, carnations, and backing materials.'),
(15,'Spring Collection',
'Vibrant baskets with fresh seasonal tulips, daisies, and wildflowers.'),
(18,'Indoor Flora',
'Long-lasting indoor plants and exotic arrangements for home decoration.');
```

```

-- 4. PRODUCTS
INSERT INTO products
(product_id, product_name, description, image_url, price, stock_quantity, category_id)
VALUES
(101,'Crimson Red Roses',
'12 long-stemmed premium red roses assembled with baby breath.',
'https://cdn.capitalflowers.com/red-roses.jpg', 45.00, 150, 10),
(102,'White Lilies Bunch',
'Hand-tied arrangement of 6 premium fresh white Asiatic lilies.',
'https://cdn.capitalflowers.com/white-lilies.jpg', 60.00, 80, 12),
(103,'Yellow Tulip Basket',
'Wicker basket containing 20 bright yellow imported Dutch tulips.',

```

```

'https://cdn.capitalflowers.com/yellow-tulips.jpg', 55.00, 45, 15),
(104,'Midnight Orchid Deluxe',
'Exotic arrangement of rare deep purple orchids with seasonal foliage.',
'https://cdn.capitalflowers.com/orchids.jpg', 85.00, 20, 10),
(105,'Sunflower Surprise',
'Cheerful bunch of 10 fresh sunflowers with green seasonal fillers.',
'https://cdn.capitalflowers.com/sunflowers.jpg', 40.00, 60, 15),
(106,'Peace Lily Pot',
'Indoor peace lily in a decorative ceramic pot, ideal as a gift.',
'https://cdn.capitalflowers.com/peace-lily.jpg', 70.00, 35, 18);

-- 5. ORDERS
INSERT INTO orders (order_id, order_date, user_id, order_status, total_amount) VALUES
(1001,'2026-06-25 10:15:00', 2,'Delivered', 150.00),
(1002,'2026-06-26 15:40:00', 3,'Processing', 180.00),
(1003,'2026-06-27 09:10:00', 2,'Pending', 155.00),
(1004,'2026-06-28 11:30:00', 4,'Delivered', 85.00),
(1005,'2026-06-29 08:45:00', 5,'Processing', 125.00);

-- 6. ORDER_ITEMS
INSERT INTO order_items (order_id, product_id, quantity, unit_price, subtotal) VALUES
(1001,101, 2, 45.00, 90.00),
(1001,102, 1, 60.00, 60.00),
(1002,102, 3, 60.00, 180.00),
(1003,101, 1, 45.00, 45.00),
(1003,103, 2, 55.00, 110.00),
(1004,104, 1, 85.00, 85.00),
(1005,105, 2, 40.00, 80.00),
(1005,106, 1, 70.00, 70.00); -- Note: total_amount covers 80+45 sample

-- 7. PAYMENTS
INSERT INTO payments
(payment_id, order_id, payment_date, payment_method, amount, payment_status)
VALUES
(881, 1001,'2026-06-25 10:16:30','Credit Card', 150.00,'Completed'),
(882, 1002,'2026-06-26 15:42:12','EasyPaisa', 180.00,'Completed'),
(883, 1003,'2026-06-27 09:10:00','Cash on Delivery', 155.00,'Pending'),
(884, 1004,'2026-06-28 11:31:00','Credit Card', 85.00,'Completed'),
(885, 1005,'2026-06-29 08:46:00','EasyPaisa', 125.00,'Pending');

-- 8. CONTACTS
INSERT INTO contacts
(contact_id, sender_name, sender_email, subject, message_body, submitted_at)
VALUES
(501,'Mian Mansha','mansha@business.com',
'Bulk Corporate Event Inquiry',
'Seeking a quotation for 50 table arrangements for seminar on July 15th.',
'2026-06-28 11:00:00'),
(502,'Sara Ali','sara@testmail.com',
'Delivery Range Question',
'Do you offer midnight deliveries to the outskirts of Islamabad?',
'2026-06-29 16:15:35'),
(503,'Ahmed Raza','ahmed@mail.com',
'Custom Bouquet Request',
'I need a custom red and white mixed bouquet for a wedding on July 5th.',
'2026-06-29 18:00:00');

```

PART 3: Analytical Queries

```
-- QUERY 1: FULL CATALOG STATUS VIEW
SELECT p.product_id, p.product_name, c.category_name,
p.price, p.stock_quantity
FROM products p
JOIN categories c ON p.category_id = c.category_id
ORDER BY c.category_name, p.product_name;

-- QUERY 2: CRITICAL LOW STOCK WARNINGS (stock < 50)
SELECT product_name, stock_quantity, price
FROM products
WHERE stock_quantity < 50
ORDER BY stock_quantity ASC;

-- QUERY 3: COMPREHENSIVE ORDER LOGS
SELECT o.order_id, o.order_date, u.name AS customer_name,
o.order_status, o.total_amount
FROM orders o
JOIN users u ON o.user_id = u.user_id
ORDER BY o.order_date DESC;

-- QUERY 4: TRANSACTION LINE-ITEMS FOR ORDER 1001
SELECT p.product_name, oi.quantity, oi.unit_price, oi.subtotal
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
WHERE oi.order_id = 1001;

-- QUERY 5: UNRESOLVED PENDING TRANSACTIONS
SELECT o.order_id, u.name AS customer_name, u.phone,
o.total_amount, p.payment_method, p.payment_status
FROM orders o
JOIN users u ON o.user_id = u.user_id
JOIN payments p ON o.order_id = p.order_id
WHERE o.order_status = 'Pending'
AND p.payment_status = 'Pending';

-- QUERY 6: REVENUE BY CATEGORY
SELECT c.category_name,
SUM(oi.subtotal) AS total_revenue_earned,
SUM(oi.quantity) AS total_items_sold
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
JOIN categories c ON p.category_id = c.category_id
JOIN orders o ON oi.order_id = o.order_id
WHERE o.order_status != 'Cancelled'
GROUP BY c.category_id, c.category_name
ORDER BY total_revenue_earned DESC;

-- QUERY 7: TOP SELLING PRODUCTS
SELECT p.product_name,
SUM(oi.quantity) AS total_units_sold,
SUM(oi.subtotal) AS gross_sales
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_id, p.product_name
ORDER BY total_units_sold DESC;
```

```
-- QUERY 8: DAILY CONSOLIDATED GROSS METRICS
```

```
SELECT DATE(order_date) AS sales_date,  
COUNT(order_id) AS total_orders_placed,  
SUM(total_amount) AS daily_gross_revenue  
FROM orders  
WHERE order_status != 'Cancelled'  
GROUP BY DATE(order_date)  
ORDER BY sales_date DESC;
```

```
-- QUERY 9: CUSTOMER ORDER HISTORY
```

```
SELECT u.name, u.email, COUNT(o.order_id) AS total_orders,  
SUM(o.total_amount) AS lifetime_value  
FROM users u  
JOIN orders o ON u.user_id = o.user_id  
GROUP BY u.user_id, u.name, u.email  
ORDER BY lifetime_value DESC;
```

```
-- QUERY 10: PAYMENT METHOD BREAKDOWN
```

```
SELECT payment_method,  
COUNT(*) AS total_transactions,  
SUM(amount) AS total_collected,  
payment_status  
FROM payments  
GROUP BY payment_method, payment_status  
ORDER BY total_collected DESC;
```

TASK 4: DATABASE IMPLEMENTATION

Capital Flowers — Islamabad Flower Shop

The complete database implementation for the Capital Flowers — Islamabad Flower Shop Management System was executed on MySQL 8.0+ using the InnoDB engine. Below is a summary of the implemented schema and sample data verification.

Component	Details	Status
Database Name	capital_flowers_dbs	Created
Total Tables	8 (users, admins, categories, products, orders, order_items, payments, carts)	Created
Total Records	5 users, 1 admin, 4 categories, 6 products, 5 orders, 8 order items, 3 payments, 3 carts	Inserted
Indexes	idx_products_name, idx_orders_user, idx_payments_order	Optimized
Constraints	Primary keys, Foreign keys, Unique constraints, Check constraints	Enforced
Normalization	3NF achieved across all 8 tables	Verified
Analytical Queries	10 queries covering catalog, orders, payments, revenue, and customer analytics	Tested & Valid
Engine	MySQL 8.0+ InnoDB with utf8mb4_unicode_ci charset	Configured